🏷️ AprilTag ID Sorting

1. Course Overview

The function implemented in this lesson can be simply understood as: **camera recognizes tags, robotic arm grasps wooden blocks, places them in different positions based on ID**.

The specific workflow is as follows:

1. Camera continuously scans the desktop, recognizing AprilTags (black and white square QR codes);
2. Through mathematical calculation, the tag's pixel position is converted to real X, Y, Z 3D coordinates in the robotic arm coordinate frame;
3. After pressing spacebar, the robotic arm automatically solves inverse kinematics to reach the target position and grasp the wooden block;
4. According to the recognized tag **ID number** (such as 1, 2, 3), the robotic arm rotates to different directions and places the wooden block in the corresponding bin, completing automatic sorting.

> **Note**
>
> **The tutorial uses the accompanying AprilTag props: 3*3cm AprilTag cube wooden blocks.**

---

2. Preparation and Running

This case requires opening 3 terminals to start the system sequentially:

2.1 Start Camera Driver and Kinematics Solver Environment

Run in **Terminal 1** (unified launch of camera driver, robotic arm driver, and inverse solver service):

```
ros2 launch dofbot_pro_driver A1pro_camera_arm.launch.py
```

```
jetson@yahboom:~$ ros2 launch dofbot_pro_driver A1pro_camera_arm.launch.py
[INFO] [launch]: All log files can be found below /home/jetson/.ros/log/2026-09-02-11-46-38-905529-yahboom-405308
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [camera_launch-1]: process started with pid [405339]
[INFO] [kinematics_dofbot-2]: process started with pid [405341]
[INFO] [Ackman_driver_A1P-3]: process started with pid [405343]
[camera_launch-1] [INFO] [launch]: All log files can be found below /home/jetson/.ros/log/2026-09-02-11-46-39-756455-yahboom-405339
[camera_launch-1] [INFO] [launch]: Default logging verbosity is set to INFO
[camera_launch-1] [INFO] [usb_cam_node_exe-1]: process started with pid [405420]
[camera_launch-1] [usb_cam_node_exe-1] libEGL warning: DRI2: failed to authenticate
[camera_launch-1] [usb_cam_node_exe-1] [INFO] [1788320800.420808348] [usb_cam]: camera_name value: test_camera
[camera_launch-1] [usb_cam_node_exe-1] [WARN] [1788320800.421007330] [usb_cam]: framerate: 120.000000
[camera_launch-1] [usb_cam_node_exe-1] [INFO] [1788320800.437814592] [usb_cam]: camera calibration URL: package://usb_cam/config/cam
era_info.yaml
[camera_launch-1] [usb_cam_node_exe-1] [WARN] [1788320800.439589882] [usb_cam]: [test_camera] does not match narrow_stereo in file /
home/jetson/yahboomcar_ros2_ws/software/library_ws/install/usb_cam/share/usb_cam/config/camera_info.yaml
[camera_launch-1] [usb_cam_node_exe-1] [INFO] [1788320800.513308203] [usb_cam]: Starting 'test_camera' (/dev/video0) at 640x480 via
mmap (mjpeg2rgb) at 120 FPS
[camera_launch-1] [usb_cam_node_exe-1] [swscaler @ 0xaaaaead862a0] No accelerated colorspace conversion found from yuv422p to rgb24.
[camera_launch-1] [usb_cam_node_exe-1] This device supports the following formats:
[camera_launch-1] [usb_cam_node_exe-1]  Motion-JPEG 1280 x 720 (60 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  Motion-JPEG 1920 x 1080 (30 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  Motion-JPEG 1024 x 768 (30 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  Motion-JPEG 640 x 480 (120 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  Motion-JPEG 800 x 600 (60 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  Motion-JPEG 1280 x 1024 (30 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  Motion-JPEG 320 x 240 (120 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  YUYV 4:2:2 1280 x 720 (9 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  YUYV 4:2:2 1920 x 1080 (6 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  YUYV 4:2:2 1024 x 768 (6 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  YUYV 4:2:2 640 x 480 (30 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  YUYV 4:2:2 800 x 600 (20 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  YUYV 4:2:2 1280 x 1024 (6 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  YUYV 4:2:2 320 x 240 (30 Hz)
[Ackman_driver_A1P-3] [INFO] [1788320800.583398163] [driver_node]: A1 driver initialized: imu_link=imu_link, pub_hz=20.0, arm_status
_hz=5.0, cmd_vel_timeout=0.50s, pub_odom=True
[camera_launch-1] [usb_cam_node_exe-1] [INFO] [1788320801.307733758] [usb_cam]: Setting 'white_balance_temperature_auto' to 1
[camera_launch-1] [usb_cam_node_exe-1] [INFO] [1788320801.307884299] [usb_cam]: Setting 'exposure_auto' to 3
[camera_launch-1] [usb_cam_node_exe-1] [INFO] [1788320801.318393433] [usb_cam]: Setting 'focus_auto' to 0
[camera_launch-1] [usb_cam_node_exe-1] [INFO] [1788320801.394173087] [usb_cam]: Timer triggering every 8 ms
```

2.2 Start Robotic Arm Desktop Grasping Action Execution Node

Run in **Terminal 2**:

```
# Depth camera version (Nuwa)
ros2 run dofbot_pro_driver grasp_desktop

# Or USB camera version
ros2 run dofbot_pro_usb grasp_usb
```

```
jetson@yahboom:~$ ros2 run dofbot_pro_usb grasp_usb
{'x_offset': 0.006563908425614734, 'y_offset': -0.004325960959468644, 'z_offset': 0.0}
----------------------------
x_offset:  0.006563908425614734
y_offset:  -0.004325960959468644
z_offset:  0.0
----------------------
init done
self.CurEndPose:  [0.0938259179959246, 0.0002628454756523636, 0.12303978979136244, -4.4890920346
31593e-06, 1.3962632447969856, -2.8289145258287586e-05]
```

2.3 Start AprilTag ID Detection and Recognition Master Node

Run in **Terminal 3**:

```
# Depth camera version (Nuwa)
ros2 run dofbot_pro_driver apriltag_detect

# Or USB camera version
ros2 run dofbot_pro_usb apriltag_usb_detect --ros-args -p tag_size:=0.02
```

The tag_size:=0.02 parameter for USB camera refers to the real edge length of the AprilTag.

```
jetson@yahboom:~$ ros2 run dofbot_pro_usb apriltag_usb_detect --ros-args -p tag_size:=0.02
{'x_offset': 0.006563908425614734, 'y_offset': -0.004325960959468644, 'z_offset': 0.0}
----------------------------
x_offset:  0.006563908425614734
y_offset:  -0.004325960959468644
z_offset:  0.0
--- USB Grasping Node Initializing ---
[INFO] [1788321022.225363974] [ApriltagUSBDetect_node]: Retrieved tag_size: 0.02 m
[INFO] [1788321022.294029245] [ApriltagUSBDetect_node]: AprilTag USB 3D vision grasping node
started successfully!
```

Visualization diagram:



3. Operation Demo and Interaction Flow

1. Place the AprilTag wooden block directly below the robot camera's field of view;
2. The recognition window will draw a green quadrilateral outline, center point, and ID label for the AprilTag;
3. **Trigger automatic grasping and sorting**:
    - Keep the image window active, press the **spacebar (`Space`)**;
    - **Adaptive distance adjustment and descent**: If the wooden block is within the robotic arm's

optimal working range, the arm directly descends precisely to grasp; if the distance is slightly off, the chassis will automatically coordinate to fine-tune forward/backward to the optimal grasping point;
- **Multi-ID differentiated unloading**: After successful grasping, the robotic arm rotates to specified target positions based on tag ID (e.g., ID:1 places in left bin, ID:2 in right bin, ID:3 in rear bin) to unload, then automatically resets to the initial patrol pose.

---

4. Core Source Code Analysis

The entire system is completed by two nodes working in collaboration, like the division of labor between a "scout" and an "action team":

Camera Type	Scout (Detection, Localization, Chassis Scheduling)	Action Team (IK Computation and Sorting Grasping)
USB Monocular Camera	`dofbot_pro_usb/apriltag_usb_detect.py`	`dofbot_pro_usb/grasp_usb.py`
3D Depth Camera	`dofbot_pro_driver/apriltag_detect.py`	`dofbot_pro_driver/grasp_desktop.py`

Regardless of which camera is used, the overall sorting logic is consistent: `Image Acquisition -> Recognize AprilTag -> Calculate 3D Coordinates -> Chassis Fine-tune Alignment -> Robotic Arm IK Descent Grasping -> Rotate and Deliver to Bin Based on ID`.

The main technical differences between the two are **distance measurement method** and **spatial compensation dimension**:

Comparison Dimension	USB Monocular Camera Solution	3D Depth Camera Solution (Nuwa-HP60C)
Distance Measurement Principle	Pinhole geometry, calculate depth from prior size	Hardware-level depth sensing, directly index true distance from aligned depth map
Spatial Compensation	Superimpose X, Y 2D deviation compensation	Synchronously superimpose X, Y, Z 3D deviation compensation
Gripper Alignment	Default fixed-angle grasping	Calculate joint 5 rotation from tag inclination angle for parallel surface grasping

---

4.1 USB Monocular Camera Core Source Code Analysis

1. Pinhole Imaging Model Physical Depth Calculation

The USB camera lacks hardware distance measurement capability. The program calculates depth from the pixel size occupied by the tag in the image and the known real edge length (`tag_size`) using the pinhole imaging formula:

```
# apriltag_usb_detect.py —— Pinhole model depth calculation
```

```
def compute_depth_pinhole(self, corners, fx, fy):
    # Calculate average pixel width and height of tag in image
    width_px  = 0.5 * (np.linalg.norm(p1 - p0) + np.linalg.norm(p2 - p3))
    height_px = 0.5 * (np.linalg.norm(p2 - p1) + np.linalg.norm(p3 - p0))

    # Depth = focal length × real size / pixel size
    z_from_w = fx * self.tag_size / width_px
    z_from_h = fy * self.tag_size / height_px
    depth_raw = 0.5 * (z_from_w + z_from_h)

    # Superimpose scale factor depth_scale and intercept depth_bias obtained during
calibration
    depth = depth_raw * self.depth_scale + self.depth_bias
    return float(depth)
```

2. Hand-Eye Transform and Chassis PID Coordination

After calculating 3D coordinates in the camera frame, they are transformed to the base frame through a preset hand-eye matrix. The chassis uses PID controller to fine-tune forward/backward based on measured horizontal distance, keeping the wooden block at the optimal 190mm working zone:

```
# apriltag_usb_detect.py —— Chassis PID coordination fine-tuning
if abs(dist_detect - self.cen_dist) > 5.5:
    # When deviation exceeds 5.5mm, chassis continues to move smoothly
    self.move_dist(dist_detect, cx)
else:
    # Distance is in position, publish target spatial pose to grasping node
    self.pos_info_pub.publish(tag_msg)
```

3. Inverse Grasping and ID-Based Rotation Sorting

The grasping node calls inverse kinematics (IK) to drive six joints for descent grasping. After successful grasping, differentiated rotation unloading is executed based on tag ID:

```
# grasp_usb.py —— Rotate and deliver to bin based on ID
if self.cur_tagId == 1:
    self.down_joint = [180, 35, 70, 5, 60, self.joint6]
elif self.cur_tagId == 2:
    self.down_joint = [180, 35, 60, 0, 90, self.joint6]
elif self.cur_tagId == 3:
    self.down_joint = [0, 30, 70, 2, 90, self.joint6]
elif self.cur_tagId == 4:
    self.down_joint = [0, 43, 48, 6, 90,self.joint6]

self.pubSixArm(self.down_joint)
time.sleep(2.5)
self.pubSingleArm(6, 30, 2000)  # Open gripper to unload
```

4.2 3D Depth Camera Core Source Code Analysis

1. RGB-D Soft Timestamp Alignment and Physical Depth Direct Index

The depth camera solution uses `ApproximateTimeSynchronizer` to ensure strict synchronization between color frames and depth point clouds. After detecting an AprilTag, the true physical distance at the center pixel is directly extracted from the depth map matrix:

```
# apriltag_detect.py —— Direct physical distance index from depth map
cz = depth_image_info[int(center_y), int(center_x)] / 1000.0  # Convert mm to meters

# Quick inverse pinhole transform to camera frame 3D point (Xc, Yc, Zc)
x = (cx - cx_param) * cz / fx
y = (cy - cy_param) * cz / fy
z = cz
```

2. Adaptive Gripper Rotation Angle Calculation (`compute_joint5`)

To ensure the gripper can tightly grip the surface regardless of how the wooden block is placed, the depth camera node uses the coordinate difference vector between two adjacent tag corners to calculate the block's deflection angle in real-time and convert it to joint 5 rotation target:

```
# apriltag_detect.py —— Calculate adaptive gripper rotation angle
vx = int(target_tag.corners[0][0]) - int(target_tag.corners[1][0])
vy = int(target_tag.corners[0][1]) - int(target_tag.corners[1][1])
target_joint5 = compute_joint5(vx, vy)

# Publish joint 5 pose angle to grasping node for execution
self.joint5.data = int(target_joint5)
self.TargetJoint5_pub.publish(self.joint5)
```

3. Hand-Eye 3D Compensation and Full Spatial Computation

After receiving 3D position, the grasping node performs cascaded matrix transformation through hand-eye matrix `EndToCamMat` and current robotic arm forward kinematics matrix `EndPointMat`, automatically adding the 3D compensation saved during calibration:

```
# grasp_desktop.py —— Hand-eye transform and 3D physical compensation overlay
PoseEndMat = np.matmul(self.EndToCamMat, self.xyz_euler_to_mat(camera_location, (0, 0, 0)))
EndPointMat = self.get_end_point_mat()
WorldPose = np.matmul(EndPointMat, PoseEndMat)

pose_T, _ = self.mat_to_xyz_euler(WorldPose)

# Superimpose 3D calibration compensation (X forward/backward, Y left/right, Z height)
pose_T[0] = pose_T[0] + self.x_offset
pose_T[1] = pose_T[1] + self.y_offset
pose_T[2] = pose_T[2] + self.z_offset
```

4. IK Execution and Multi-Bin Automatic Classification

After obtaining the precise spatial pose with compensation, the grasping node asynchronously initiates IK request, drives chassis and robotic arm coordinated descent grasping, and performs multi-station flow placement based on tag ID:

```
# grasp_desktop.py —— IK initiation and bin placement
request = ArmKinemarics.Request()
request.tar_x = pose_T[0]
request.tar_y = pose_T[1]
request.tar_z = pose_T[2]
request.pitch = 1.3962  # Keep gripper vertical downward
request.kin_name = "ik"
future = self.client.call_async(request)
```

Through this software-hardware layered design, whether you are equipped with a highly cost-effective USB monocular lens or a high-precision 3D depth camera, the system can automatically adapt to the best computation strategy for stable autonomous sorting closed-loop.

4.3 Grasping Offset Tuning (Both Camera Models)

If position deviation occurs during actual grasping (such as missing, knocking over objects, offset left/right, or hitting the table), you can directly modify the spatial offset configuration file for the corresponding camera:

1. Where to Modify

Configuration files are located in the `~/yahboomcar_ros2_ws/dofbot_pro_ws/src/dofbot_pro_driver/config/` directory. Select based on camera type:

- **3D Depth Camera (Nuwa-HP60C)**: `offset_value.yaml`
- **USB Camera**: `usb_offset_value.yaml`

2. How to Modify

The file contains offset parameters in X, Y, Z directions (unit: meters, recommended adjustment per tweak is `0.005`~`0.01`, i.e., 5~10mm):

- `x_offset` **(forward/backward direction, positive toward vehicle front)**:
  - Gripper extends too far and knocks over object **decrease value** (adjust smaller or set negative)
  - Gripper doesn't extend far enough and misses **increase value** (adjust larger)
- `y_offset` **(left/right direction, positive toward vehicle left)**:
  - Gripper deviates right **increase value** (correct left)
  - Gripper deviates left **decrease value** (correct right)
- `z_offset` **(up/down height, positive toward ceiling)**:
  - Gripper too high in air and can't grasp **decrease value** (probe deeper)
  - Gripper hits or scrapes table **increase value** (raise higher)